MATERIAL DIDÁCTICO · ACCESO A DATOS

TUTORIAL RESUELTO 05 · UNIDAD 02

Un componente Java reutilizable

Interfaz pública, implementación JDBC, instalación Maven y consumo desde otro proyecto.

Objetivo y resultado

Crear una biblioteca Java reutilizable que exponga una interfaz de acceso a clubes y que otro proyecto consuma como dependencia Maven. Seguimos usando el servidor MySQL de los tutoriales anteriores.

Paso 1. Revisar la estructura

ESTRUCTURA

```
T05_Componente_Reutilizable/
├── componente/
│   ├── pom.xml
│   └── src/main/java/es/curso/ad/componente/
│       ├── Club.java
│       ├── RepositorioClubes.java
│       └── RepositorioMysql.java
└── consumidor/
    ├── pom.xml
    └── src/main/java/es/curso/ad/consumidor/Main.java
```

El consumidor conoce la interfaz y utiliza una clase de implementación. Separar proyectos permite actualizar la biblioteca sin copiar sus fuentes dentro de cada programa.

Paso 2. Crear la biblioteca componente

2.1. pom.xml del componente

Ruta: **Codigo/T05_Componente_Reutilizable/componente/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>componente-clubes</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>com.mysql</groupId>
      <artifactId>mysql-connector-j</artifactId>
      <version>8.4.0</version>
    </dependency>
  </dependencies>
</project>
```

2.2. Club.java

Ruta: **Codigo/T05_Componente_Reutilizable/componente/src/main/java/es/curso/ad/componente/Club.java**

ARCHIVO COMPLETO

```
package es.curso.ad.componente;

public record Club(int id, String nombre, String ciudad) { }
```

2.3. RepositorioClubes.java

Ruta: **Codigo/T05_Componente_Reutilizable/componente/src/main/java/es/curso/ad/componente/RepositorioClubes.java**

ARCHIVO COMPLETO

```
package es.curso.ad.componente;

import java.sql.SQLException;
import java.util.List;
import java.util.Optional;

public interface RepositorioClubes {
    List<Club> listar() throws SQLException;
    Optional<Club> buscar(int id) throws SQLException;
}
```

2.4. RepositorioMysql.java

Ruta: **Codigo/T05_Componente_Reutilizable/componente/src/main/java/es/curso/ad/componente/RepositorioMysql.java**

ARCHIVO COMPLETO

```
package es.curso.ad.componente;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

public class RepositorioMysql implements RepositorioClubes {
    private final String url;
    private final String usuario;
    private final String clave;

    public RepositorioMysql(String url, String usuario, String clave) {
        this.url = url;
        this.usuario = usuario;
        this.clave = clave;
    }

    @Override
    public List<Club> listar() throws SQLException {
        List<Club> resultado = new ArrayList<>();
        try (Connection con = DriverManager.getConnection(url, usuario, clave);
```

```
             PreparedStatement st = con.prepareStatement(
                     "SELECT id, nombre, ciudad FROM club ORDER BY id");
             ResultSet filas = st.executeQuery()) {
            while (filas.next()) {
                resultado.add(construir(filas));
            }
        }
        return resultado;
    }

    @Override
    public Optional<Club> buscar(int id) throws SQLException {
        try (Connection con = DriverManager.getConnection(url, usuario, clave);
             PreparedStatement st = con.prepareStatement(
                     "SELECT id, nombre, ciudad FROM club WHERE id = ?")) {
            st.setInt(1, id);
            try (ResultSet filas = st.executeQuery()) {
                if (filas.next()) return Optional.of(construir(filas));
            }
        }
        return Optional.empty();
    }

    private Club construir(ResultSet filas) throws SQLException {
        return new Club(filas.getInt("id"),
                filas.getString("nombre"), filas.getString("ciudad"));
    }
}
```

La implementación usa SQL parametrizado y no imprime. El constructor recibe URL, usuario y contraseña, con lo que no están fijos dentro del componente.

Paso 3. Publicar el JAR en el repositorio local

Arranca MySQL si no lo has hecho. En la terminal de la carpeta componente ejecuta:

TERMINAL

```
mvn clean install
```

Maven debe mostrar **BUILD SUCCESS**. Encontrarás el JAR bajo **target/componente-clubes-1.0-SNAPSHOT.jar** y una copia en el repositorio Maven local (~/.m2/repository). No se copia el JAR manualmente a la carpeta del consumidor.

Paso 4. Crear el proyecto consumidor

4.1. pom.xml del consumidor

Ruta: **Codigo/T05_Componente_Reutilizable/consumidor/pom.xml**

ARCHIVO COMPLETO

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>es.curso.ad</groupId>
  <artifactId>consumidor-clubes</artifactId>
  <version>1.0-SNAPSHOT</version>
  <properties>
    <maven.compiler.release>17</maven.compiler.release>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>
  <dependencies>
    <dependency>
      <groupId>es.curso.ad</groupId>
      <artifactId>componente-clubes</artifactId>
      <version>1.0-SNAPSHOT</version>
    </dependency>
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>3.5.0</version>
        <configuration><mainClass>es.curso.ad.consumidor.Main</mainClass></configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

4.2. Main.java del consumidor

Ruta: **Codigo/T05_Componente_Reutilizable/consumidor/src/main/java/es/curso/ad/consumidor/Main.java**

ARCHIVO COMPLETO

```
package es.curso.ad.consumidor;

import es.curso.ad.componente.Club;
import es.curso.ad.componente.RepositorioClubes;
import es.curso.ad.componente.RepositorioMysql;
import java.sql.SQLException;

public class Main {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3307/ad_ud2?connectionTimeZone=UTC";
        RepositorioClubes repositorio =
                new RepositorioMysql(url, "ad_user", "ad_pass");
        try {
            for (Club club : repositorio.listar()) {
                System.out.println(club);
            }
            System.out.println("ID 2: " + repositorio.buscar(2).orElse(null));
```

```
        } catch (SQLException e) {
            System.err.println("Error del componente: " + e.getMessage());
        }
    }
}
```

El Main declara una variable RepositorioClubes. Puede elegir otra implementación sin cambiar el código que recorre los resultados.

Paso 5. Ejecutar y revisar dependencias

Abre la terminal dentro de consumidor y ejecuta:

TERMINAL

```
mvn clean compile exec:java
mvn dependency:tree
```

El primer comando lista los clubes y busca el ID 2. El segundo muestra cómo Maven resuelve la dependencia componente-clubes y su conector JDBC transitivo. Si el consumidor no encuentra el componente, repite mvn clean install en la carpeta componente y comprueba que groupId, artifactId y version coinciden exactamente.

Paso 6. Pruebas antes de R05

- Desconecta MySQL y verifica que SQLException llega al consumidor sin que la biblioteca finja un resultado.
- Busca el ID 99999 y comprueba que recibes Optional.empty().
- Explica la diferencia entre ejecutar mvn package y mvn install.
- Añade una segunda implementación del contrato en la relación, sin modificar Main más allá de la instancia elegida.

Componente bien diseñado
Una biblioteca de acceso a datos no debe leer Scanner ni imprimir por pantalla. Sus métodos proporcionan resultados y errores que otra aplicación pueda tratar.